BCSCHOOL

Empowering Digital Skills For The Jobs Of The Future

by

BC Soft
consulenza sistemi informativi

AiRED
Artificial Intelligence Ricerca e Sviluppo

JAVA JEE
-
Componenti fondamentali di un programma Java

TEACHER

Vincenza Ruvolo

vincenza.ruvolo@bcsoft.net

Sommario

- Le basi della programmazione ad oggetti (OOP)
- I metodi in Java
- Le variabili in Java
- I metodi costruttori
- Introduzione ai package
- Altri componenti fondamentali della programmazione Java

Le basi della programmazione Object Oriented

I concetti alla base della conoscenza del linguaggio Java sono:

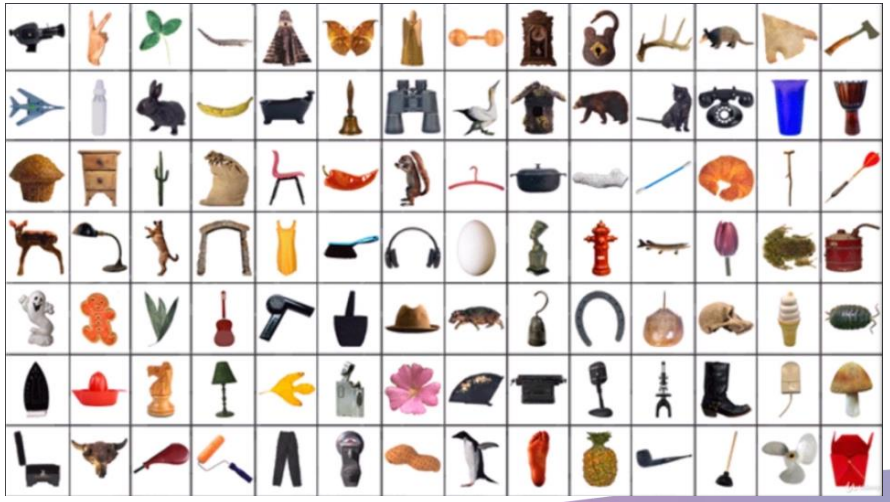
- Classe
- Oggetto
- Variabile
- Metodo
- Costruttore
- Package

Gli oggetti

Gli oggetti sono gli elementi individuali che si è in grado di conoscere, comprendere e con i quali, in determinati casi, è possibile anche interagire.

Gli oggetti

In generale sono le entità sulle quali è possibile indirizzare il pensiero, indipendentemente dal fatto che sono identità materiali (es. un albero), immateriale (es. un conto corrente), oppure puramente immaginari (es. l'unicorno).



Gli oggetti

Come è possibile riconoscere un oggetto?

Osservando gli oggetti: auto e albero.

Cosa consente di rilevarli ed identificarli?

Astrazione: processo spontaneo e inconsapevole.

Astrazione

È un processo che viene eseguito talvolta in maniera inconsapevole dagli esseri umani.

Funziona nel seguente modo: a partire da una serie di fenomeni distinti, si mantengono le informazioni che sembrano importanti per arrivare ad un concetto.

Il concetto

Un concetto è un'idea, modello generale, che si utilizza in modo quasi automatico per unificare la comprensione di oggetti diversi, che si sono formati nella mente attraverso l'esperienza e un processo di astrazione.

P.e. la videocamera.

Che cos'è un OGGETTO?

Un oggetto è un'entità qualunque, alla quale si è in grado di applicare un concetto che consente di riconoscere la natura dell'oggetto.

La classificazione

La classificazione è il tipo di astrazione che si esegue per associare ad un oggetto di un determinato tipo, di determinata natura, un certo concetto.

Processo di classificazione

Dato un oggetto individuale, per esempio una videocamera, che possiede un insieme di proprietà generiche:

Forma, colore, peso, dimensione, anche proprietà più specifiche, e funzioni.

Processo di classificazione

Osservando 3 videocamere differenti, è possibile classificarle applicando il principio di astrazione in base alle proprietà, o meglio come istanze della classe videocamera.

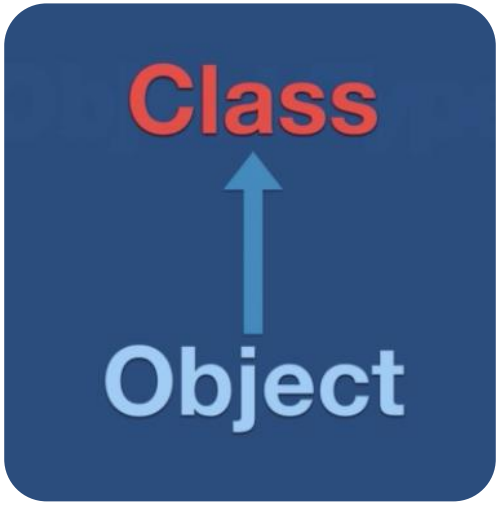
Istanza di concetto

Un oggetto è un’istanza di un concetto.

Dato un determinato concetto, tutti gli oggetti che si possono classificare come esempi validi di questo concetto, sono detti istanze di questo concetto.

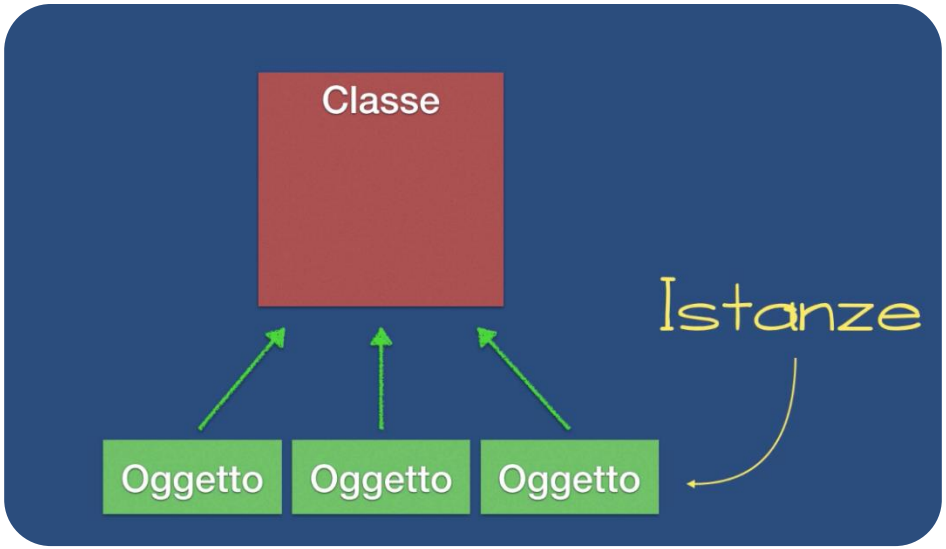
Concetto VS Oggetto

La differenza tra Concetto e Oggetto nei Sistemi Software Object Oriented viene espressa più propriamente come differenza tra Object e Object Type (Tipo di Oggetto) chiamati anche Classi.



La classe

La classe è la forma generale per la creazione di un insieme di oggetti, dove questi oggetti sono tutti dello stesso tipo (o istanze della stessa classe).



La classe

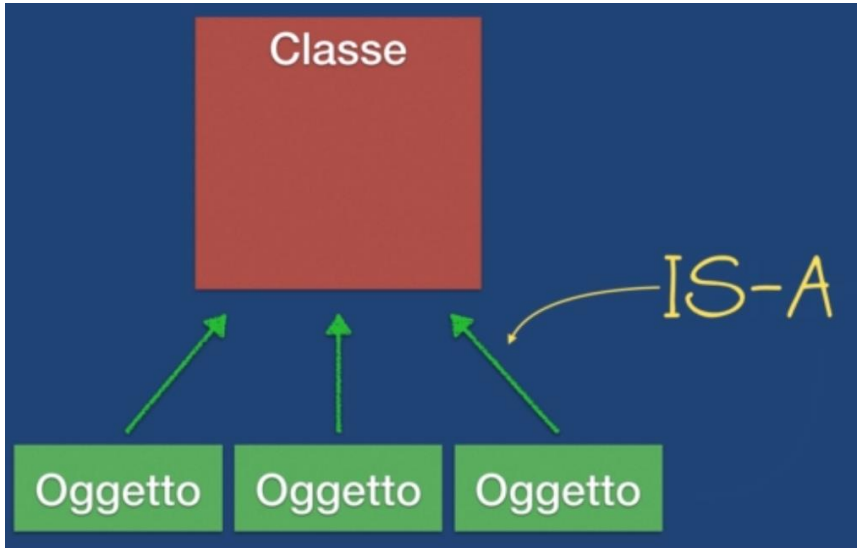
Gli oggetti che vengono creati a partire da una classe vengono detti Istanze di questa classe.

Non esiste un limite per la creazione di istanze di una classe.

Un oggetto è un’istanza di una classe.

Relazione istanza-classe

La relazione che si viene a creare tra le istanze e la classe (classificazione) è detta anche **IS-A** (è un).



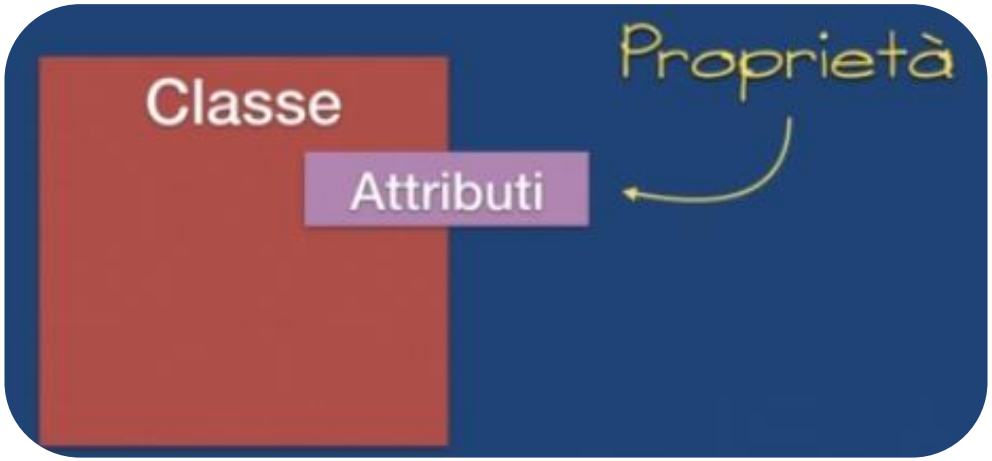
Le proprietà

Le proprietà sono le caratteristiche che contraddistinguono i concetti cioè i tipi di oggetto e quindi le classi.

Esse sono caratteristiche possedute da tutti gli oggetti di una determinata Classe.

La classe

Una classe consente di definire un insieme di attributi (di proprietà) che nel loro insieme determinano quelle che sono le informazioni che servono per rappresentare ogni singola istanza di una certa classe.



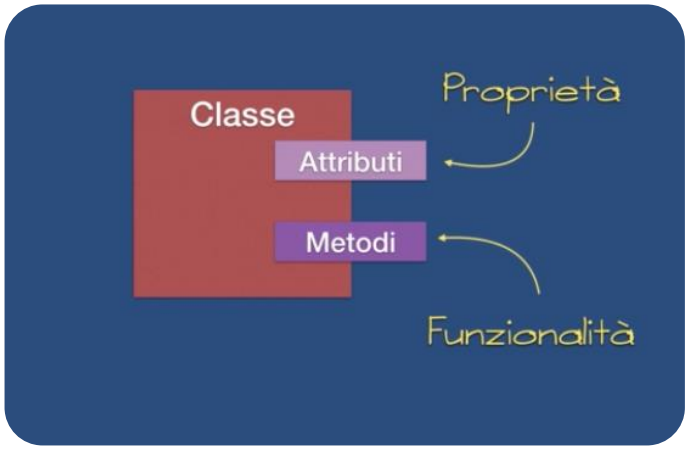
La classe

I concetti vengono creati a partire da un insieme di comportamenti che sono specifici degli oggetti che possono essere considerati sempre istanze di un determinato concetto, ovvero di una classe.

La classe

All'interno di una classe è possibile definire un insieme di metodi (o funzioni), che conterranno al loro interno tutte istruzioni necessarie a svolgere determinati compiti.

ATTRIBUTI + METODI = MEMBRI DELLA CLASSE



I metodi

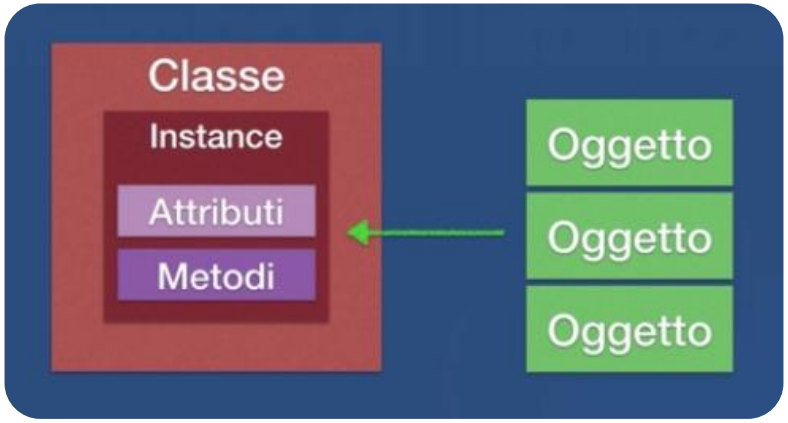
Il **metodo** rappresenta l'azione che può compiere un oggetto.

I metodi di una classe possono essere **pubblici** e quindi vengono usati dagli oggetti di un'altra classe per formare l'interfaccia pubblica della classe, o **privati** e quindi invisibili ad altre classi.

Membri di istanza e di classe

È possibile modellare 2 tipi di attributi e di metodi a seconda che questi appartengono alla classe o alle singole istanze della classe.

Gli attributi e i metodi, sono membri che vengono assegnati a ciascuna istanza di una classe.



Ricapitolando

Una classe rappresenta una descrizione di una categoria di oggetti, individuati da comportamenti e caratteristiche simili.

I comportamenti vengono definiti **“metodi”** mentre le caratteristiche vengono definite **“proprietà”.**

Ricapitolando

Una classe è una famiglia di oggetti.

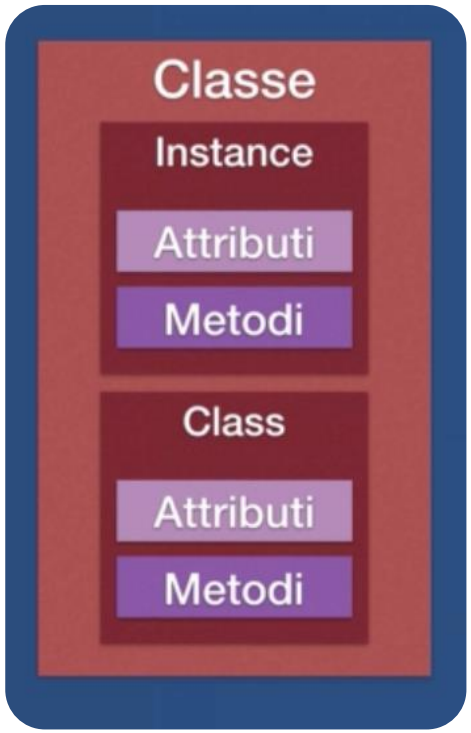
Ogni classe è definita da attributi che determinano le distinzioni tra i vari oggetti.

Ricapitolando

Una classe può avere in dotazione due categorie differenti di membri:

- Gli attributi e i metodi che si riferiscono alle singole istanze della classe.

- Gli attributi e i metodi che si riferiscono alla classe in quanto tale



Ricapitolando

Ad esempio se si ha una classe Cane, gli attributi di tale classe possono essere:

- La razza
- Il colore
- La lunghezza del pelo
- Ecc.

Ricapitolando

Gli attributi sono registrati in variabili il cui valore può variare da un oggetto all’altro, per tale motivo sono definite variabili d’istanza.

Ricapitolando

Oltre agli attributi gli oggetti di una classe dispongono di comportamenti. Ritornando alla classe Cane i comportamenti (metodi) possono essere:

- Nasce
- Cresce
- Mangia
- Caccia
- Ecc.

Le basi della programmazione Object Oriented: classi ed oggetti

Dal punto di vista pratico, si definisca la classe Punto:

TestOggettiPunto.java | Punto.java

```
1  package punto;
2
3  public class Punto {
4
5      public int x;
6      public int y;
7
8  }
9
```

Le basi della programmazione Object Oriented: classi ed oggetti

Lo scopo di questa classe è definire il concetto di punto (a due dimensioni) tramite la definizione delle sue coordinate su di un piano cartesiano.

Le basi della programmazione Object Oriented: classi ed oggetti

Le coordinate x e y sono state astratte mediante due attributi (dette solitamente variabili) dichiarati di tipo intero (int), quindi potranno assumere solo valori di tipo intero.

Le basi della programmazione Object Oriented: classi ed oggetti

Le variabili x e y sono state dichiarate pubbliche (public), il che significa che potrebbero essere utilizzate anche da altre classi diverse dalla classe Punto.

Le basi della programmazione Object Oriented: classi ed oggetti

È possibile salvare questo listato in un file chiamato:

Punto.java

e compilarlo tramite il comando:

javac Punto.java

ottenendo così il file Punto.class.

Le basi della programmazione Object Oriented: classi ed oggetti

Questa volta però non sarà possibile mandarlo in esecuzione tramite il comando:

java Punto

In quanto si otterrebbe il seguente messaggio di errore:

```
Errore: il metodo principale non è stato trovato nella
classe Punto.
Definire il metodo principale come:
    public static void main(String[] args)
```

Le basi della programmazione Object Oriented: classi ed oggetti

Ciò accade perché in questa classe è assente il metodo main().

Se si esegue il comando

java Punto

la JVM prova a cercare questo metodo per eseguirlo e non trovandolo termina istantaneamente l'esecuzione.

Le basi della programmazione Object Oriented: classi ed oggetti

Con il codice scritto fino ad ora, non è stato ancora definito nessun punto vero e proprio.

Occorre quindi definire gli oggetti punto, ovvero le creazioni fisiche realizzate a partire dalla definizione data dalla classe, quindi bisogna istanziare degli oggetti.

Come istanziamo un oggetto in Java?

Istanziare, è il termine object oriented che si usa in luogo di creare fisicamente, ed il risultato di un'istanziazione viene chiamato oggetto.

L'operatore che permette di istanziare un oggetto a partite dalla classe è l'operatore **new**.

Operatore New

L’operatore new è alla base di Java. Permette di creare un’istanza a partire da una classe. L’istanza sarà memorizzata nello heap della memoria principale.

Il new per creare l’oggetto deve invocare il **costruttore** della classe che si desidera istanziare.

Costruttore

Abbiamo detto che una classe è composta da una serie di proprietà o attributi, ed una serie di comportamenti o metodi.

Il «costruttore» è un metodo speciale che ha lo stesso nome della classe in cui è contenuto, ma senza modificatore, permette di inizializzare una istanza della classe.

Può avere nessuno o più parametri.

Costruttore

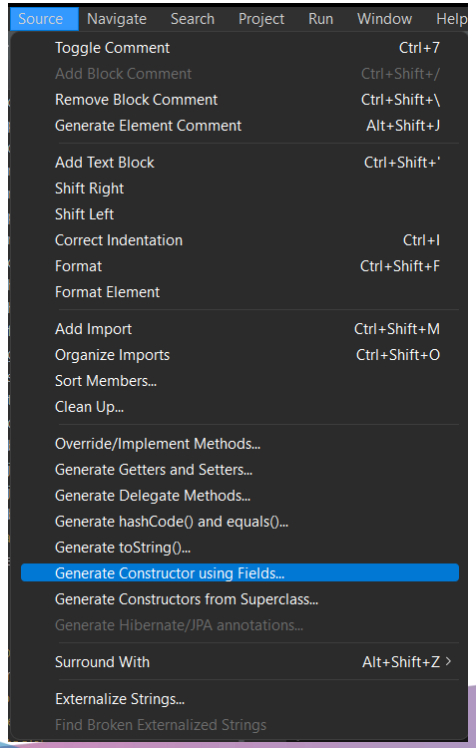
Se non scritto in modo esplicito dal programmatore Java aggiunge automaticamente un costruttore detto di **default** che permette di creare le istanze tramite la sintassi: Classe oggetto= new Classe();

Le parentesi vuote di new Classe() stanno ad indicare che si è richiamato il costruttore di default.

Se il costruttore di default viene scritto esplicitamente dal programmatore prende il nome di **costruttore senza parametri o vuoto**.

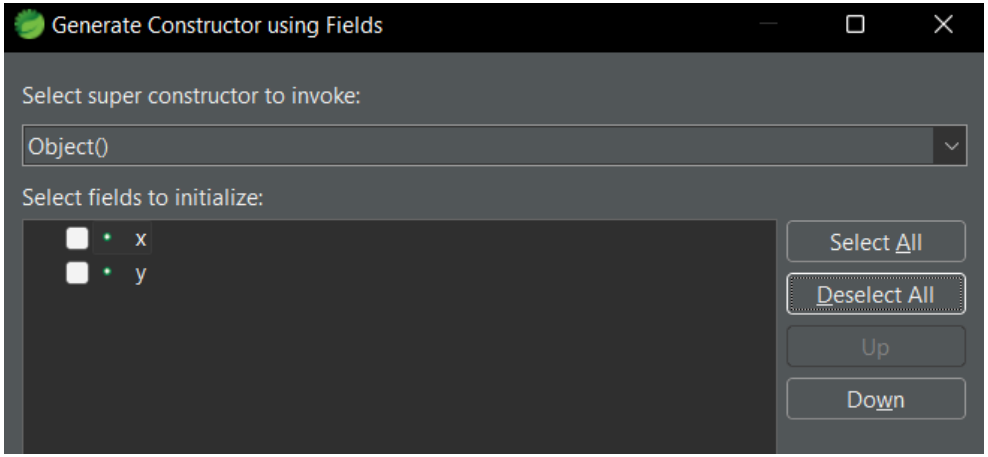
Costruttore

Possiamo comunque scrivere il costruttore in modo esplicito «a mano» oppure tramite le funzionalità dell'Ide.



Costruttore

Nel menù possiamo selezionare il costruttore senza parametri o con parametri, semplicemente aggiungendo o rimuovendo i flag opportuni, ad esempio non selezionando le proprietà avremo un costruttore vuoto:




```
public class Punto {

    public int x;
    public int y;


    public Punto() {
        super();
    }
```

Cliccare su «Generate»

Costruttore

Se invece selezioniamo entrambi gli attributi otterremo un costruttore con parametri

```
public Punto(int x, int y) {
    super();
    this.x = x;
    this.y = y;
}
```

Come richiamo un costruttore con parametri?

```
package punto;

public class Main {

    public static void main(String[] args) {

        //chiamato il costruttore senza parametri
        //inizializziamo gli attributi
        Punto punto1= new Punto();

        punto1.x=1;
        punto1.y=2;

        //con il costruttore con parametri basta una istruzione
        Punto punto2= new Punto(1,2);

    }

}
```

Le basi della programmazione Object Oriented: classi ed oggetti

Tornando all'esempio della classe Punto, saranno istanziati oggetti un'altra classe che contiene un metodo main() che chiameremo **TestOggettiPunto:**

TestOggettiPunto.java | Punto.java

```java
package punto;

public class TestOggettiPunto {

    public static void main(String[] args) {

        Punto punto1;
        punto1 = new Punto();
        punto1.x = 2;
        punto1.y = 6;

        Punto punto2;
        punto2 = new Punto();
        punto2.x = 0;
        punto2.y = 1;

        System.out.println("punto 1: (" + punto1.x + ", " + punto1.y +")");
        System.out.println("punto 2: (" + punto2.x + ", " + punto2.y +")");
    }

}
```

Le basi della programmazione Object Oriented: classi ed oggetti

Una riga di codice viene solitamente chiamata **statement** e può contenere una o più istruzioni.

All'interno del metodo main() si trovano gli statement che devono essere eseguiti, dalla riga 7 alla riga 18.

Le basi della programmazione Object Oriented: classi ed oggetti

Alla riga 7 si incontra la prima istruzione da seguire, nonché la dichiarazione di un **oggetto** di **tipo Punto** che viene chiamato punto1.

Le basi della programmazione Object Oriented: classi ed oggetti

Con l’istruzione alla riga 8 si sta creando l’oggetto punto1 della classe Punto.

Con le ultime righe si vanno a stampare le coordinante dei punti.

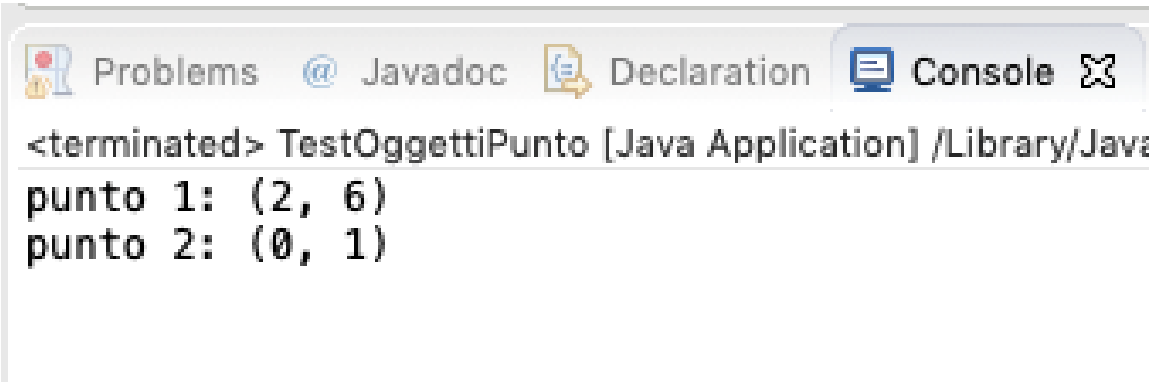
Le basi della programmazione Object Oriented: classi ed oggetti

println → stampa e va accapo

È possibile incorporare il tutto utilizzando la concatenazione (+)

Ad esempio:

```
System.out.println("punto 1: (" + punto1.x + ", " + punto1.y +")");
```



Introduzione ai package

Un package in Java permette di raggruppare in un’unica entità complessa classi Java logicamente correlate. Fisicamente il package non è altro che una cartella (sinonimo di directory) del sistema operativo.

Attenzione: non tutte le cartelle sono package.

Introduzione ai package

Se volessimo far appartenere la classe Punto a un package geometria, dovremmo semplicemente inserire come prima istruzione del nostro file sorgente la seguente dichiarazione di package:

```
package geometria;

public class Punto
{
   //codice omesso
}
```

Introduzione ai package

Questo comporterebbe che il file compilato (.class) dovrà risiedere in una cartella chiamata proprio geometria.

Introduzione ai package

Se si desidera creare un albero di package più strutturato è possibile usare la sintassi sfruttando l'operatore dot (che anche in questo caso significa appartenenza):

package studio.matematica.geometria;

Introduzione ai package

In questo caso il file compilato risiederà all’interno di una cartella geometria a sua volta contenuta in una cartella matematica a sua volta contenuta in una cartella studio.

Introduzione ai package

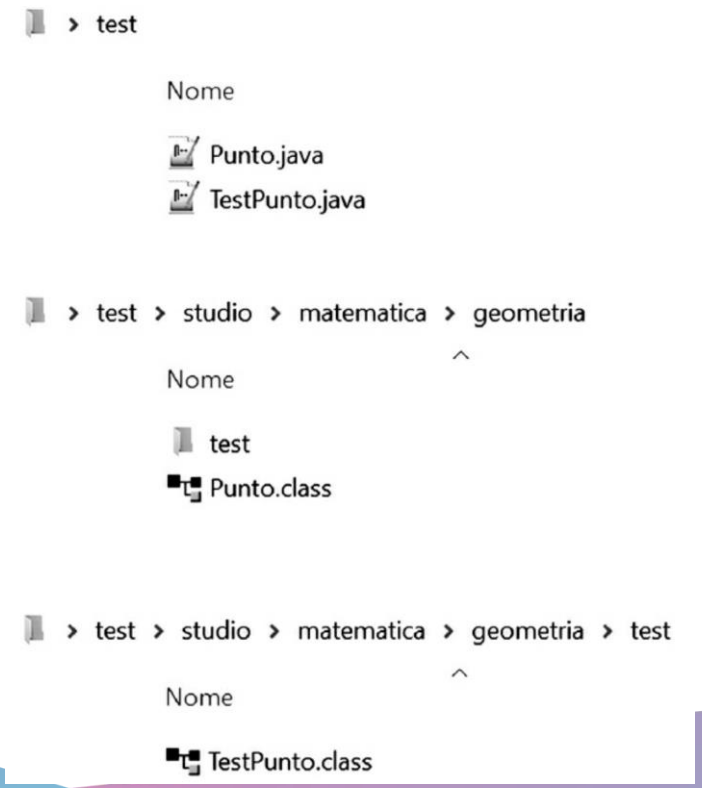
Se una classe chiamata TestPunto vuole usare la classe Punto e non si trova nello stesso package **studio.matematica.geometria,** dovrà importare la classe Punto con il comando import, che va dichiarato dopo la dichiarazione di package, ma prima della dichiarazione della classe, come nel seguente codice:

Introduzione ai package

```
package studio.matematica.geometria.test;

import studio.matematica.geometria.Punto;

public class TestPunto {

        public static void main (String args[]) {
                Punto p = new Punto();
                . . .
    }
}
```

Introduzione ai package

Da un punto di vista di folder si avrà:



Introduzione ai package

Dal prompt dei comandi, è possibile utilizzare la seguente sintassi per eseguire la compilazione, dopo essersi posizionati nella cartella test:

javac -d . Punto.java TestPunto.java

-d (abbreviativo di -directory) utilizzato per specificare di voler creare le directory a partire da cartella corrente (.)

Introduzione ai package

Per eseguire il file **TestPunto** invece, sempre dalla cartella test bisogna lanciare il comando:

java studio.matematica.geometria.test.TestPunto

I moduli

I moduli sono la novità più importante di Java 9, consentono di gestire la struttura delle applicazioni in maniera più pulita e funzionale, divisa in cartelle.

I moduli

Queste possono contenere package, classi ed altre risorse e un file descrittore.

I moduli

All’interno della cartella che deve rappresentare il modulo ci deve essere un file .java chiamato per convenzione module-info.java che dichiara il nome e le caratteristiche del modulo.

I moduli

Per esempio con la seguente sintassi:

```
module com.cdsc.test {
    exports com.cdsc.test;
}
```

I moduli

Grazie a questo codice si sta dichiarando un modulo di nome **com.cdsc.test**.

Esso dichiara che siano visibili al suo esterno tutte le classi appartenenti al package com.cdsc.test.

I moduli sono utili soprattutto per applicazioni molto grandi, che hanno bisogno di essere organizzate al meglio.

Inizializzatori

Inizializzatori

Gli inizializzatori sono componenti che vanno dichiarati internamente alle classi. Vengono utilizzati per inizializzare la classe o l’oggetto.

Inizializzatori

Esistono sia quelli di classe (detti anche inizializzatori statici) che quelli di istanza.

Inizializzatori

Hanno la caratteristica di essere automaticamente invocati una sola volta nel momento in cui viene caricata la classe (inizializzatori statici) o l'oggetto (inizializzatori d'istanza).

Inizializzatori

Come esempio si creerà un inizializzatore d’istanza.

```
public class TestInizializzatore {

   {
      System.out.println("Questo è un inizializzatore d'istanza");
   }

}
```

Domande e approfondimenti